Intelligent Robotics Assignment 1

ROS2 — Navigation and Sensing for TurtleBot3 (waffle)

GitHub Repository Videos Drive

Ferracane Salvatore* Vianello Giacomo†

1 Introduction and Overview

This report presents our solution to *Assignment 1*, which required the development of a complete navigation pipeline for the TurtleBot3 in simulation. The objective was to enable the robot to detect AprilTags, compute a safe intermediate goal between them, and subsequently identify cylindrical tables within the environment. In addition, the system was required to temporarily suspend Nav2 when entering a corridor, activate a custom LiDAR-based local controller for corridor traversal, and then resume global navigation once the corridor had been successfully exited.

Our architecture adopts a hybrid *PLAN, SENSE–ACT* paradigm (see Fig. 1). In this formulation, the planning layer plays a supervisory role: it defines navigation goals, monitors contextual conditions, and coordinates transitions between controllers. This paradigm is particularly suitable for our scenario, as once the planner computes the intermediate goal between the AprilTags, the robot must be able to detect when it enters a corridor and promptly suspend the ongoing Nav2 navigation in order to switch to a custom, hand-crafted LiDAR-based controller. A high-level overview of the main system components is provided in the following subsections.

In summary, this document offers a concise overview of the overall architecture and its core components. A detailed description of each node's functionality, as well as the full implementation, is available in the official GitHub repository, while a Google Drive folder containing some videos can be found here.

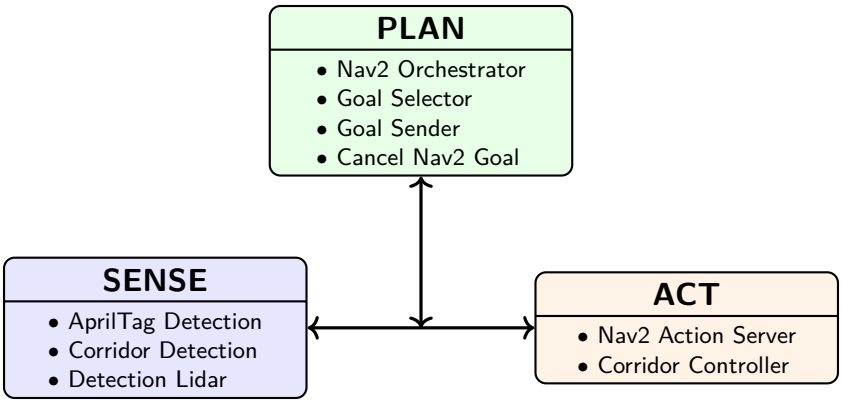


Figure 1: Hybrid architecture: the Plan layer supervises the reactive Sense–Act loop. Each block contains the ROS 2 nodes developed to achieve the final navigation pipeline.

*salvatore.ferracane@studenti.unipd.it
†giacomo.vianello.2@studenti.unipd.it

1.1 SENSE

The *SENSE* block provides the perceptual layer of the system, integrating data from two complementary sources: the robot's onboard LiDAR and a fixed overhead camera in Gazebo. Together, these sensors supply the information needed to interpret the environment and support navigation.

This layer combines global and local perspectives. The external camera is used to detect AprilTags and compute global reference points that serve as navigation goals. In parallel, the onboard LiDAR enables local geometric reasoning by (i) identifying corridor structures to trigger controller switching and (ii) detecting cylindrical tables to estimate their position relative to the robot. The sensing layer is implemented through three ROS 2 nodes (see Fig. 2).

SENSE

- **AprilTag Detection**: extracts AprilTags from the fixed-camera stream to compute global navigation goals.
- **Corridor Detection**: analyzes LiDAR scans to detect corridor structures, determine corridor entry and exit.
- **Detection LiDAR**: processes LiDAR data to identify cylindrical tables and track their relative position.

Figure 2: SENSE block: high-level nodes functionality.

1.2 PLAN

The *PLAN* block handles high-level decision-making and coordination. It interprets information from the sensing layer, selects appropriate navigation goals, and manages transitions between global and local navigation behaviors.

Rather than following a fixed sequence, this layer implements a dynamic supervision logic. It first verifies that the Nav2 stack is fully initialized before issuing any navigation commands. Once ready, it computes a safe goal position from the detected AprilTags and dispatches it to Nav2. Throughout execution, the planning layer monitors the robot's context: when a corridor is detected, it preempts the global planner to avoid conflicts and temporarily hands control over to the custom corridor controller. After corridor traversal, it seamlessly restores the previously active goal and resumes global navigation. The planning layer is implemented through four ROS 2 nodes (see Fig. 3).

PLAN

- **Nav2 Orchestrator**: checks Nav2 readiness and authorizes goal dispatch only once the navigation stack is fully operational.
- **Goal Selector**: computes a safe goal between the detected AprilTags and exposes it through a service.
- **Goal Sender**: queries the selected goal and sends it to Nav2 via an action; it also pauses and resumes goal dispatching based on corridor status.
- **Cancel Nav2 Goal**: listens for corridor activation events and cancels any ongoing Nav2 goal to allow the corridor controller to take over.

Figure 3: PLAN block: high-level nodes functionality.

1.3 ACT

The *ACT* block constitutes the execution layer of the system, responsible for carrying out the navigation decisions produced by the planning layer. It implements a hybrid control strategy that switches between global navigation and reactive local control depending on the robot's context.

In open environments, the robot relies on the standard Nav2 stack to perform global path planning, obstacle avoidance, and motion control toward the desired pose. When the robot enters a corridor, however, this behavior is temporarily superseded by a dedicated reactive controller that uses real-time LiDAR data to keep the robot centered between the walls and guide it safely through the passage. Once the corridor is exited, control is seamlessly handed back to Nav2. The acting layer

consists of two ROS 2 nodes (see Fig. 4).

ACT

- **Nav2 Action Server**: standard Nav2 component responsible for executing global navigation goals.
- **Corridor Controller**: a reactive controller that overrides Nav2 during corridor traversal, generating velocity commands based on LiDAR-derived wall geometry.

Figure 4: ACT block: high-level nodes functionality.

2 UML: A (quite) Deeper Explaination

Figure 5 presents a UML diagram that illustrates the interactions among the ROS 2 nodes and the flow of information through topics, services, and actions. According to the legend, topics are shown in violet, services in green, and actions in orange. Each square block corresponds to a node developed for this project, unless explicitly stated (e.g., the AprilTag node provided by the standard package).

The overall execution pipeline unfolds as follows. The *AprilTag* package continuously processes images from the fixed camera and publishes the detected markers in the camera frame. In parallel, the *Nav2 Orchestrator* node initializes and monitors the Nav2 navigation stack until all its components are fully active. Once Nav2 is ready, the *Goal Sender* node requests a navigation target from the *Goal Selector* node through a service call. The Goal Selector computes a suitable reference pose located between the detected AprilTags, after transforming their positions into the map frame. When the Goal Sender receives the service response, it forwards the corresponding goal to Nav2 via an action request. From this moment, Nav2 handles global planning, obstacle avoidance, and motion execution to steer the TurtleBot from its initial pose—specified with an associated covariance—toward the computed goal.

During navigation, the *Corridor Detector* node continuously analyzes LiDAR data to identify corridor structures. When a corridor is detected, the node publishes a Boolean flag on the /corridor_active topic. This triggers the *Cancel Nav2 Goal* node, which immediately cancels the ongoing Nav2 action, prevents the Goal Sender from issuing new goals, and notifies the reactive *Corridor Controller*. This controller takes direct control of the robot, using a robust enhanced PD strategy to keep the robot centered between the walls until the corridor ends. When the Corridor Detector determines that the robot is no longer in a corridor, it resets the Boolean flag. The Goal Sender then resubmits the previously computed goal to Nav2, which resumes global navigation until the robot reaches the target region between the two AprilTags. At this point, the Goal Sender shuts down to avoid issuing further commands.

Finally, once the robot reaches the goal, it prints the locations of all detected obstacles (tables). Since only two of the three tables are visible from the goal region, the *Detection Lidar* node incorporates a dedicated memory logic that records obstacles the moment they are first detected, ensuring complete information is reported at the end of the run.

For a clearer understanding of the full workflow, including a detailed description of each node, we suggest the reader to refer to the accompanying demonstration video and the project's GitHub repository.

3 Conclusion

As shown in the attached video, all requirements of Assignment 1 have been fulfilled. Nonetheless, several extensions are possible. Both the *Corridor Detection Node* and the *Corridor Controller Node* can benefit from more advanced methodologies. Corridor estimation, in particular, could be strengthened by incorporating filtered or enhanced RANSAC fitting, enabling reliable wall extraction even when the corridor is not perfectly linear. On the control side, a more structured approach based on the analysis of the error dynamics and on the design of a dedicated controller would improve tracking performance and robustness.

During simulations, particularly when observing in Rviz2, we noticed a key limitation: the *odom* frame oscillates relative to the world frame (map). This behavior likely stems from the uncertainty in the robot's initial pose, expressed through the covariance matrix Σ defined in the launch file. Because all measurements and reference frames are linked to odometry, this instability propagates into obstacle and wall detection as well as control actions, increasing overall uncertainty. Such oscillations are expected in realistic scenarios, since odometry and sensor readings are inherently affected by noise. However, their impact could be mitigated by introducing filtering techniques within the perception pipeline, thereby improving robustness and reliability.

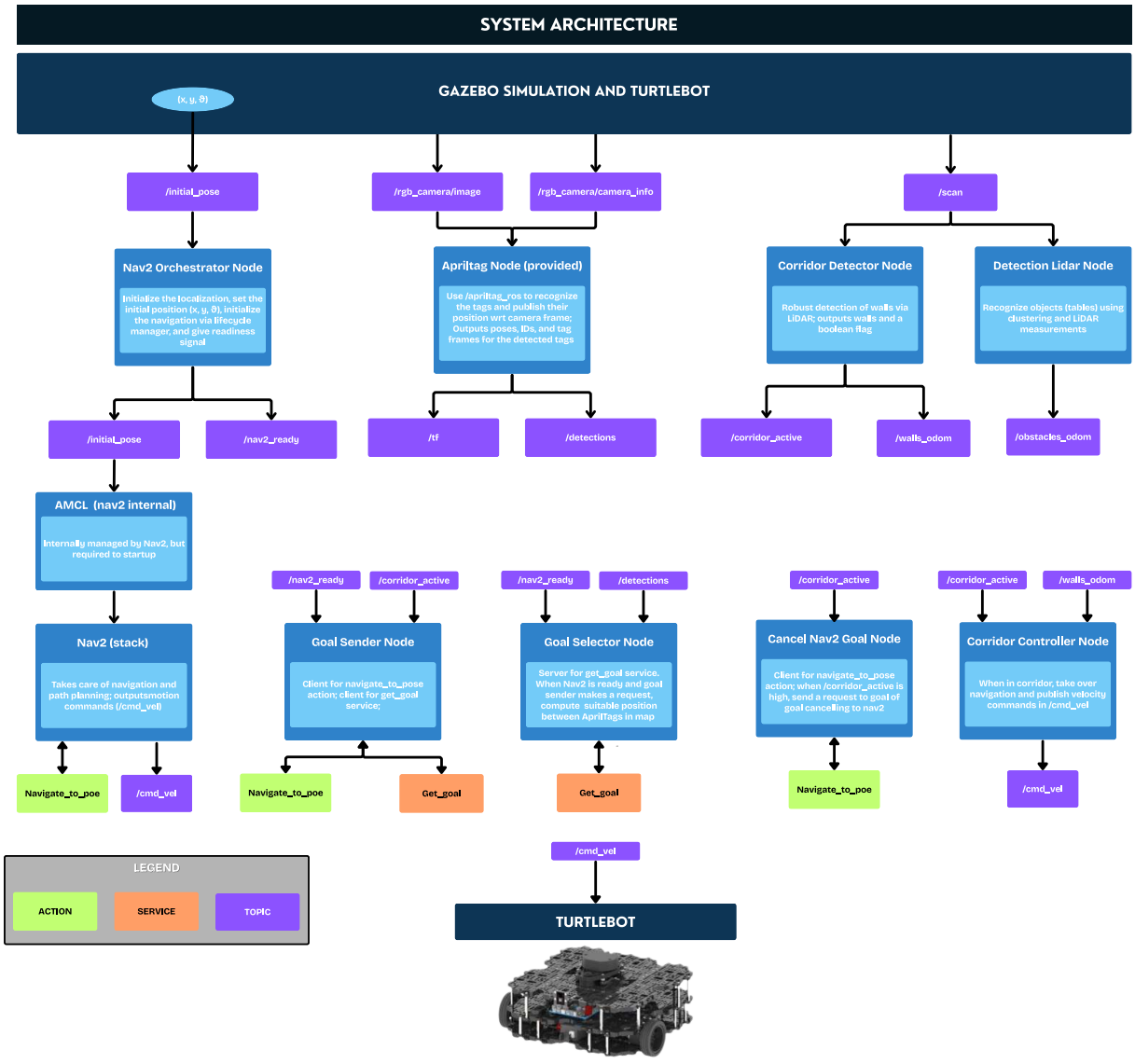


Figure 5: Unified Modeling Language (UML) diagram of the project.